

2.10 Modules ^

Settings

Beta features

script

Thus, a programmer will typically write Python code in a file, and then run the file from the command line to the interpreter. Such a file is called a script.

module

A solution is to use a module, which is a file containing Python code that can be imported and used by scripts, other modules, or the interactive interpreter.

import

To import a module means to execute the code contained by the module, making the definitions within that module available for use by the importing program.

PARTICIPATION ACTIVITY

2.10.1: A module is a file containing Python statements and definitions that can be imported into a Python program.

Start ☐ 2x speed

```
def fct():  
    # ...  
  
def sq():  
    # ...
```

```
def fct():  
    # ...  
  
def sq():  
    # ...
```

```
def fct():  
    # ...  
def sq():  
    # ...
```

funcs.py

```
x = fct() * sq()
# ...
```

script1.py

```
x = fct() / sq()
# ...
```

script2.py

```
import funcs
```

```
x = funcs.fct() *
    funcs.sq()
```

script1.py

```
import
```

```
x = f
```

sc

Captions ^

1. A programmer writes scripts containing functions and code using those functions. Multiple functions.
2. The functions can instead be defined in another file. The file can be imported as a "module"

dependency

A module being required by another program is often called a dependency.

sys.modules

A dictionary of the loaded modules is stored in `sys.modules` (available in the `sys` library module).

module object

A module object is simply a namespace that contains definitions for...

PARTICIPATION ACTIVITY

2.10.3: Importing a module.

Start



2x speed

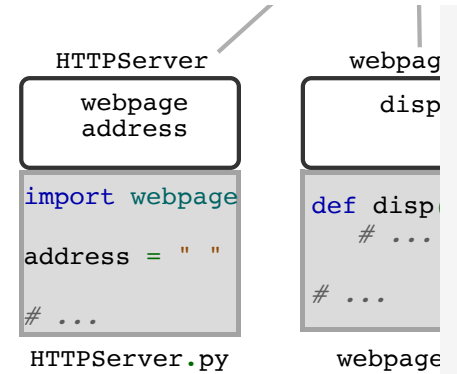
```
import HTTPServer
import webpage

my_ip = HTTPServer.address
```

sys.mo

```
HTTPSe
webpp
```

```
webpage.disp(my_ip)
# ...
```



Captions ^

1. `sys.modules` checks for `HTTPServer`. A new module object is created. The module is then imported.
2. `HTTPServer`'s code is executed in module namespace. `sys.modules` checks for `webpage`. The module is then imported and inserted in `sys.modules`.
3. `webpage`'s code is executed in module namespace. `webpage` is added to `HTTPServer` namespace.
4. `HTTPServer`'s code continues executing.
5. `webpage` has already been imported. Existing module is loaded.

Figure 2.10.1: Contents of `my_funcs.py`.

```
def factorial(num):
    """Calculates and returns the factorial (num!)"""
    x = 1
    for i in range(1, num + 1):
        x *= i

    return x
```

Figure 2.10.2: Using factorial from my_funcs.py.

```
import my_funcs

n = int(input("Enter number: "))
fact = my_funcs.factorial(n)

for i in range(1, n + 1):
    print(i, end=" ")
    if i != n:
        print("*", end=" ")

print(f"= {fact}")
```

```
Enter number: 5
1 * 2 * 3 * 4 * 5
...
Enter number: 3
1 * 2 * 3 = 6
```

**CHALLENGE
ACTIVITY**

2.10.1: Enter the output of modules.

763650.1443318.qx3zqy7

Start

Type the program's output

main.py

arithmetic.py

```
import arithmetic

def calculate(number):
    return number * 4

print(arithmetic.calculate(2))
print(calculate(2))
```

6

8

1

2

3

4

Check**Next**View solution ▼ *(Instructors only)*

**CHALLENGE
ACTIVITY**

2.10.2: Modules.

763650.1443318.qx3zqy7

Start

String `customer_name` and integer `luggage_quantity` are read from input. Module `airline_kiosk` defines `deposit()`. Import modules `airline_kiosk` and `airline_luggage` into `main.py`.

[Click here for example](#) ▼**main.py****airline_kiosk.py****airline_luggage.py**

```
1
2 """ Your code goes here """
3
4 customer_name = input()
5 luggage_quantity = int(input())
6
7 airline_kiosk.check_in(customer_name, luggage_quantity)
8 airline_luggage.deposit(luggage_quantity)
```

1

2

Check**Next level**[View solution](#) ▼ *(Instructors only)*